Modified LeNet-5 for CIFAR-10 Image Classification

Homework Assignment 4

Course: Spring 2026 Introduction to Deep Learning
Name: Karim Smires
Due Date: April 20, 2026

Abstract

This report presents a modified LeNet-5 implementation for CIFAR-10 colorful image classification using PyTorch. Three network variants are evaluated: a baseline model without Dropout or Batch Normalization, a model with an added Dropout layer, and a model with Batch Normalization layers. All models are trained for 20 epochs using the Adam optimizer and compared using training accuracy, test accuracy, and total training time. The baseline model reaches 62.24% test accuracy, the Dropout model improves generalization to 64.34%, and the Batch Normalization model performs best with 66.47% test accuracy. All three models satisfy the assignment requirement of exceeding 50% test accuracy, with Batch Normalization providing the strongest overall improvement.

1 Introduction

The goal of this homework is to build and evaluate a modified LeNet-5 network for CIFAR-10 image classification. Unlike the original LeNet-5, which was designed for grayscale handwritten digits, CIFAR-10 contains small natural RGB images from ten object categories. This makes the classification task more challenging and requires adjusting the input layer from one channel to three channels.

In addition to implementing the baseline architecture, the assignment requires testing two common regularization and stabilization techniques: Dropout and Batch Normalization. These methods are widely used in deep learning because they can improve generalization and training behavior. The purpose of this report is to describe the three implemented models, summarize the training setup, and compare their performance on CIFAR-10.

2 Related Work

LeNet-5 [1] is one of the earliest successful convolutional neural networks and was originally proposed for handwritten digit recognition. Its stacked convolutional and fully connected design became a foundational architecture for later CNN models.

Dropout [2] is a regularization technique that randomly deactivates a subset of neurons during training. This reduces co-adaptation between features and often improves test performance by lowering overfitting.

Batch Normalization [3] normalizes intermediate activations within a mini-batch and then applies learned scale and shift parameters. It often stabilizes optimization, speeds convergence, and can provide a mild regularization effect.

3 Data Description

The CIFAR-10 dataset consists of 60,000 color images of size 32×32 drawn from ten object classes: airplane, automobile, bird, cat, deer, dog, frog, horse, ship, and truck.

- **Training set:** 50,000 images
- **Test set:** 10,000 images
- **Image shape:** $3 \times 32 \times 32$
- **Classes:** 10

Images are converted to tensors and normalized channel-wise using the standard CIFAR-10 statistics implemented in the code:

$$\mu = (0.4914, 0.4822, 0.4465), \qquad \sigma = (0.2470, 0.2435, 0.2616)$$

No data augmentation is used in this homework.

4 Method Description

4.1 Baseline Model

The baseline model is a modified LeNet-5 adapted to RGB input. It uses three convolutional layers followed by two fully connected layers and ends with a `LogSoftmax` output so that the model can be trained with negative log-likelihood loss.

4.2 Dropout Model

The Dropout variant keeps the same convolutional structure as the baseline but inserts one `nn.Dropout(p=0.5)` layer after the first fully connected layer. During training, Dropout randomly zeroes activations with probability $p = 0.5$, which helps reduce overfitting by preventing the network from relying too heavily on particular hidden units.

4.3 Batch Normalization Model

The Batch Normalization variant adds normalization layers after each convolutional layer and after the first fully connected layer. The ordering used in the code is Conv/Linear $\rightarrow$ BatchNorm $\rightarrow$ ReLU, which follows standard practice. For a mini-batch input x, Batch Normalization computes

$$\hat{x} = \frac{x - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}}, \qquad y = \gamma \hat{x} + \beta$$

where $\mu_{\mathcal{B}}$ and $\sigma_{\mathcal{B}}^2$ are the batch mean and variance, while γ and β are learnable parameters.

5 Model Description

The implemented modified LeNet-5 architecture for CIFAR-10 is summarized below.

Layer	Operation	Output Shape	Notes
Input	—	$3 \times 32 \times 32$	RGB image
C1	Conv2d(3→6, 5×5)	$6 \times 28 \times 28$	[BN] → ReLU
S2	MaxPool2d(2×2)	$6 \times 14 \times 14$	
C3	Conv2d(6→16, 5×5)	$16 \times 10 \times 10$	[BN] → ReLU
S4	MaxPool2d(2×2)	$16 \times 5 \times 5$	
C5	Conv2d(16→120, 5×5)	$120 \times 1 \times 1$	[BN] → ReLU
Flatten	—	120	
F6	Linear(120→84)	84	[BN] → ReLU → [Dropout]
F7	Linear(84→10)	10	LogSoftmax output

Table 1: Modified LeNet-5 architecture used in the homework. Bracketed layers are included only in the corresponding variant.

The three evaluated configurations are:

1. **Baseline:** no Dropout and no Batch Normalization.
2. **Dropout:** one Dropout layer after F6.
3. **BatchNorm:** BatchNorm layers after C1, C3, C5, and F6.

6 Experimental Procedure and Results

6.1 Experimental Setup

All models are trained using the same training procedure so that the comparison is fair.

Hyperparameter	Value
Optimizer	Adam
Learning rate	0.001
Batch size	64
Epochs	20
Loss function	NLLLoss
Dataset	CIFAR-10

Table 2: Shared hyperparameters used for all three models.

During each epoch, training accuracy is computed from the mini-batches seen in the training loop, and test accuracy is computed by evaluating the model on the full test set. Total wall-clock training time is also recorded for each model.

6.2 Results

Model	Train Acc	Test Acc	Time
Baseline	76.67%	62.24%	65.1 s
Dropout	72.02%	64.34%	65.0 s
BatchNorm	**82.69%**	**66.47%**	68.8 s

Table 3: Final training accuracy, final test accuracy, and training time for the three model variants.

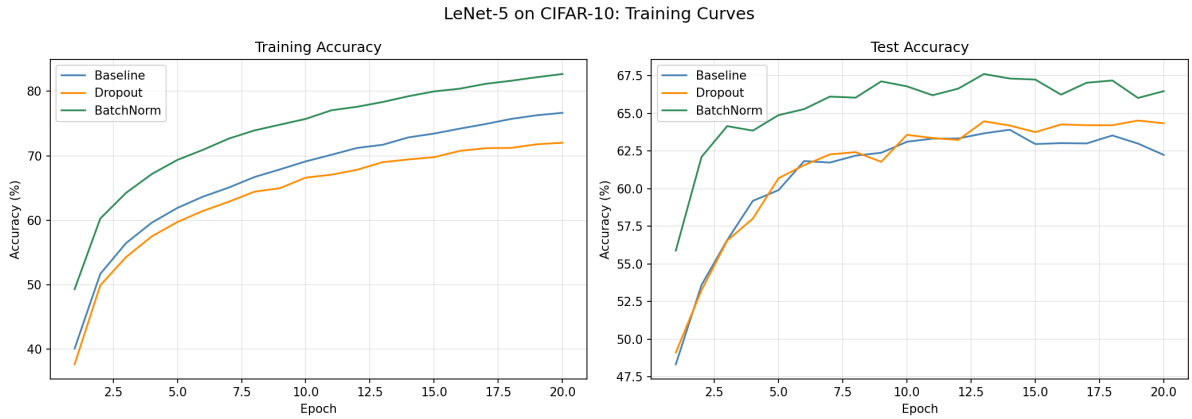

Figure 1: Training accuracy and test accuracy curves for the three LeNet-5 variants on CIFAR-10.

6.3 Analysis

All three models exceed the assignment requirement of 50% test accuracy, which shows that the modified LeNet-5 architecture is sufficient to learn useful features on CIFAR-10.

Baseline performance. The baseline model achieves 62.24% test accuracy with 76.67% training accuracy. The gap between train and test accuracy suggests moderate overfitting as training progresses.

Effect of Dropout. Adding one Dropout layer decreases the final training accuracy to 72.02% but improves the test accuracy to 64.34%. This is the expected pattern for regularization: the model fits the training data less aggressively while generalizing better to unseen data.

Effect of Batch Normalization. The BatchNorm model achieves the best overall performance, reaching 82.69% training accuracy and 66.47% test accuracy. It also shows faster early convergence in the recorded results. This suggests that Batch Normalization improves optimization and allows the network to learn stronger features without sacrificing generalization.

Overall comparison. Among the three variants, Batch Normalization provides the largest performance gain over the baseline, improving test accuracy by 4.23 percentage points. Dropout also improves generalization, but its gain is smaller at 2.10 percentage points. Based on these results, Batch Normalization is the most effective modification for this homework setting.

7 Conclusion

This homework implemented and evaluated three modified LeNet-5 models for CIFAR-10 classification. The baseline model provided a solid starting point and already exceeded the required 50% test accuracy. Adding Dropout improved generalization, while adding Batch Normalization produced the best overall result.

The final ranking is:

1. **BatchNorm**: 66.47% test accuracy
2. **Dropout**: 64.34% test accuracy
3. **Baseline**: 62.24% test accuracy

These results show that both regularization and normalization are useful for LeNet-5 on CIFAR-10, with Batch Normalization offering the strongest benefit in this experiment.

References

[1] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.

[2] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," *Journal of Machine Learning Research*, vol. 15, pp. 1929–1958, 2014.

[3] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," in *Proc. International Conference on Machine Learning (ICML)*, pp. 448–456, 2015.

A Source Code

```python
"""
Homework 4: LeNet-5 for CIFAR-10 Classification
Three variants: Baseline, Dropout, BatchNorm
"""

import time
from collections import OrderedDict

import matplotlib.pyplot as plt
import torch
import torch.nn as nn
import torchvision
import torchvision.transforms as transforms
from torch.utils.data import DataLoader

#  Device
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Using device: {device}")

```

```python
#  Hyperparameters
BATCH_SIZE = 64
EPOCHS = 20
LR = 0.001

#  Task 1: Data Loading
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.4914, 0.4822, 0.4465), (0.2470, 0.2435, 0.2616)),
])

train_dataset = torchvision.datasets.CIFAR10(
    root="./data", train=True, download=True, transform=transform
)
test_dataset = torchvision.datasets.CIFAR10(
    root="./data", train=False, download=True, transform=transform
)

train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=True, num_workers=0)
test_loader  = DataLoader(test_dataset,  batch_size=BATCH_SIZE, shuffle=False, num_workers=0)

print(f"Train samples: {len(train_dataset)}, Test samples: {len(test_dataset)}")

#  Task 2: Baseline LeNet-5 (no Dropout, no BatchNorm)
class LeNet5(nn.Module):
    def __init__(self):
        super().__init__()
        self.convnet = nn.Sequential(OrderedDict([
            ("c1",    nn.Conv2d(3, 6, kernel_size=5)),   # 36, 3228
            ("relu1", nn.ReLU()),
            ("s2",    nn.MaxPool2d(kernel_size=2, stride=2)),  # 2814
            ("c3",    nn.Conv2d(6, 16, kernel_size=5)),  # 1410
            ("relu3", nn.ReLU()),
            ("s4",    nn.MaxPool2d(kernel_size=2, stride=2)),  # 105
            ("c5",    nn.Conv2d(16, 120, kernel_size=5)), # 51
            ("relu5", nn.ReLU()),
        ]))
        self.fc = nn.Sequential(OrderedDict([
            ("f6",    nn.Linear(120, 84)),
            ("relu6", nn.ReLU()),
            ("f7",    nn.Linear(84, 10)),
            ("sig7",  nn.LogSoftmax(dim=-1)),
        ]))

    def forward(self, x):
        x = self.convnet(x)
        x = x.view(x.size(0), -1)
        x = self.fc(x)
        return x


#  Task 3: LeNet-5 with Dropout
class LeNet5Dropout(nn.Module):
    def __init__(self, p=0.5):
        super().__init__()
        self.convnet = nn.Sequential(OrderedDict([
            ("c1",    nn.Conv2d(3, 6, kernel_size=5)),
            ("relu1", nn.ReLU()),
            ("s2",    nn.MaxPool2d(kernel_size=2, stride=2)),
            ("c3",    nn.Conv2d(6, 16, kernel_size=5)),
```

```python
            ("relu3", nn.ReLU()),
            ("s4",    nn.MaxPool2d(kernel_size=2, stride=2)),
            ("c5",    nn.Conv2d(16, 120, kernel_size=5)),
            ("relu5", nn.ReLU()),
        ]))
        self.fc = nn.Sequential(OrderedDict([
            ("f6",      nn.Linear(120, 84)),
            ("relu6",   nn.ReLU()),
            ("drop6",   nn.Dropout(p=p)),
            ("f7",      nn.Linear(84, 10)),
            ("sig7",    nn.LogSoftmax(dim=-1)),
        ]))

    def forward(self, x):
        x = self.convnet(x)
        x = x.view(x.size(0), -1)
        x = self.fc(x)
        return x


#  Task 4: LeNet-5 with Batch Normalization
class LeNet5BatchNorm(nn.Module):
    def __init__(self):
        super().__init__()
        self.convnet = nn.Sequential(OrderedDict([
            ("c1",    nn.Conv2d(3, 6, kernel_size=5)),
            ("bn1",   nn.BatchNorm2d(6)),
            ("relu1", nn.ReLU()),
            ("s2",    nn.MaxPool2d(kernel_size=2, stride=2)),
            ("c3",    nn.Conv2d(6, 16, kernel_size=5)),
            ("bn3",   nn.BatchNorm2d(16)),
            ("relu3", nn.ReLU()),
            ("s4",    nn.MaxPool2d(kernel_size=2, stride=2)),
            ("c5",    nn.Conv2d(16, 120, kernel_size=5)),
            ("bn5",   nn.BatchNorm2d(120)),
            ("relu5", nn.ReLU()),
        ]))
        self.fc = nn.Sequential(OrderedDict([
            ("f6",    nn.Linear(120, 84)),
            ("bn6",   nn.BatchNorm1d(84)),
            ("relu6", nn.ReLU()),
            ("f7",    nn.Linear(84, 10)),
            ("sig7",  nn.LogSoftmax(dim=-1)),
        ]))

    def forward(self, x):
        x = self.convnet(x)
        x = x.view(x.size(0), -1)
        x = self.fc(x)
        return x


#  Training & Evaluation helpers
criterion = nn.NLLLoss()


def evaluate(model, loader):
    model.eval()
    correct = total = 0
    with torch.no_grad():
        for images, labels in loader:
            images, labels = images.to(device), labels.to(device)
            outputs = model(images)
```

```
            _, predicted = outputs.max(1)
            total += labels.size(0)
            correct += predicted.eq(labels).sum().item()
    return 100.0 * correct / total


def train_model(model, name):
    model = model.to(device)
    optimizer = torch.optim.Adam(model.parameters(), lr=LR)

    train_accs, test_accs = [], []
    start = time.time()

    for epoch in range(1, EPOCHS + 1):
        model.train()
        correct = total = 0

        for images, labels in train_loader:
            images, labels = images.to(device), labels.to(device)
            optimizer.zero_grad()
            outputs = model(images)
            loss = criterion(outputs, labels)
            loss.backward()
            optimizer.step()

            _, predicted = outputs.max(1)
            total += labels.size(0)
            correct += predicted.eq(labels).sum().item()

        train_acc = 100.0 * correct / total
        test_acc  = evaluate(model, test_loader)
        train_accs.append(train_acc)
        test_accs.append(test_acc)

        print(f"[{name}] Epoch {epoch:2d}/{EPOCHS} | "
              f"Train: {train_acc:.2f}% | Test: {test_acc:.2f}%")

    elapsed = time.time() - start
    print(f"\n[{name}] Training time: {elapsed:.1f}s | "
          f"Final train acc: {train_accs[-1]:.2f}% | "
          f"Final test acc:  {test_accs[-1]:.2f}%\n")

    return train_accs, test_accs, elapsed


#  Run all three models
results = {}

for model_cls, label in [
    (LeNet5(),          "Baseline"),
    (LeNet5Dropout(),   "Dropout"),
    (LeNet5BatchNorm(), "BatchNorm"),
]:
    train_accs, test_accs, elapsed = train_model(model_cls, label)
    results[label] = {
        "train_accs": train_accs,
        "test_accs":  test_accs,
        "time":       elapsed,
    }

#  Task 5: Plot & Compare
epochs = range(1, EPOCHS + 1)
colors = {"Baseline": "steelblue", "Dropout": "darkorange", "BatchNorm": "
```

```
    seagreen"}

fig, axes = plt.subplots(1, 2, figsize=(14, 5))
fig.suptitle("LeNet-5 on CIFAR-10: Training Curves", fontsize=14)

for label, data in results.items():
    axes[0].plot(epochs, data["train_accs"], label=label, color=colors[label])
    axes[1].plot(epochs, data["test_accs"],  label=label, color=colors[label])

for ax, title in zip(axes, ["Training Accuracy", "Test Accuracy"]):
    ax.set_xlabel("Epoch")
    ax.set_ylabel("Accuracy (%)")
    ax.set_title(title)
    ax.legend()
    ax.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig("training_curves.png", dpi=150)
plt.show()
print("Saved training_curves.png")

#  Summary table
print("\n" + "=" * 58)
print(f"{'Model':<12} {'Train Acc':>10} {'Test Acc':>10} {'Time (s)':>10}")
print("=" * 58)
for label, data in results.items():
    print(f"{label:<12} {data['train_accs'][-1]:>9.2f}% "
          f"{data['test_accs'][-1]:>9.2f}% "
          f"{data['time']:>9.1f}s")
print("=" * 58)
```

Listing 1: hw4.py — modified LeNet-5 training and evaluation